

Assignment Task 5 (Gradescope)

Component A: Snakes and Ladders (1)

- (a) the vector basically tracks how many turns are left to win from each square. the very last number represents the square just before the finishing line. rolling any number on the die (1 through 6) will either land you exactly on the finish line or overshoot it. since overshooting counts as a win, we are guaranteed to finish the game in exactly 1 move.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # constructing the transition matrix
5 P = np.zeros((10, 10))      #10x10 zero matrix
6
7 for i in range(9):
8     for die in range(1, 7):
9         target_square = i + die
10
11         #if we overshoot the 10th square, stop on the 10th square
12         if target_square > 9:
13             P[i, 9] += 1
14
15         else:
16             P[i, target_square] += 1
17
18 P[9, 9] = 6
19
20 print(P.astype(int))
21 plt.matshow(P.astype(int), cmap="viridis")
22 plt.show()
23
24 # PART 2&3
25 P_prob = P / 6.0
26
27 P_tilde = P_prob[0:9, 0:9]
28 s = np.ones((9, 1))
29 I = np.eye(9)
30
31 h = np.round(np.linalg.solve(I - P_tilde, s), 2)
32 print("h =\n", h)
33
34 print("h flattened =\n", h.flatten())
35
```

this is a code i wrote to construct the matrix & solve for h:

```
[ 3.04  2.78  2.52  2.16  1.85  1.59  1.36  1.17  1.  ]
h =
[[3.04]
 [2.78]
 [2.52]
 [2.16]
 [1.85]
 [1.59]
 [1.36]
 [1.17]
 [1.  ]]
h flattened =
[3.04 2.78 2.52 2.16 1.85 1.59 1.36 1.17 1.  ]
```

- (b) i chose :

```
matlab > putting_it_all_together.m
1 % Enter the SL vector below
2 SL = [2 16; 3 15; 4 14; 22 5; 23 6; 24 7]
3
```

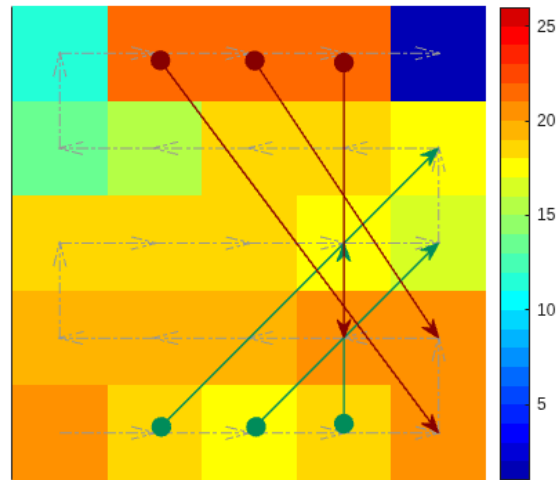
it follows all the rules: no ladders at the start, no snakes at the finish, no shared tiles or loops, and no six consecutive snakes. three parallel ladders are placed right at the start to shoot players into the top half of the board. however, this fast-track road drops them right in front of a wall of three massive, scary snakes at the top, which slide players right back down to the bottom. this traps everyone in a frustrating loop, adding spice to the game.

```
1 % Enter the SL vector below
2 SL = [2 16; 3 15; 4 14; 22 5; 23 6; 24 7]
3
4
5 % We have included some Matlab code below. You do not need to understand it all to answer the question, but
6 % we encourage you to look at the loops and ask for help if you do not see how they work. It is very common
7 % to use loops in this sort of project.
8
9 % The number of squares
10 n = 25;
11
12 % Construct the P matrix first by ignoring the snakes and ladders
13
14 % The probability of taking a step from i to j is determined by the die
15 P = zeros(n);
16 for i=1:n
17     for j=i+1:min(i+6,n)
18         P(i,j) = 1/6;
19     end
20 end
21
22 % Consider the case when the step could be greater than the number
23 % of places remaining on the board
24 for i = 1:6
25     P(n-6+i, n) = min((i+1)/6, 1);
26 end
27
28 % Adjust the probability when landing on the start of a ladder
29 % or tail of a snake
30 no_SL = height(SL);
31 for i= 1:no_SL
32     for j=1:n
33         if j == SL(i,2)
34             P(SL(i,1),j) = 1;
35         else
36             P(SL(i,1),j) = 0;
37         end
38     end
39 end
40
41 % Construct the s vector
42 s = ones([n-1 1]);
43
44 % Extract tildeP
45 tildeP = P(1:(n-1),1:(n-1));
46
47 % Solve for h below
48 I_mat = eye(n-1);
49 h = (I_mat - tildeP) \ s
50
51 % Plot the expect game duration
52 plot_SL_game(SL, h)
53
54
```

- h vector output

```
h =  
  
20.3725  
18.5647  
17.8939  
18.7647  
20.5895  
20.3461  
20.0762  
19.7460  
19.3385  
19.0851  
18.9451  
18.8857  
18.4566  
17.7647  
16.8939  
17.5647  
18.1049  
18.5293  
15.8823  
13.6134  
11.6686  
21.5895  
21.3461  
21.0762
```

- plot output



(c) the matlab output shows us that the **expected number of turns to win is 20.37 (~20 rolls) turns**. it is the very first number in the output of the h vector (highlighted in pink in the above image). the first number basically depicts position $h(1)$ where the player begins the game on square 1, representing the total average number of rolls required to go past all the obstacles and reach the finish line: square 25.

(d) in the MATLAB version, landing on the start of a snake or ladder does not move the player immediately. instead, the player spends their next turn travelling along it.

to make it the standard version of snakes and ladders, the transition matrix P would need to change so that landing on a snake/ladder square instantly sends the player to its destination in the same turn. this means:

- if square i is the start of a snake or ladder and it connects to square j ,

- then any probability of landing on i should instead be redirected straight to j .
- for example:
 - if square 4 has a ladder to square 14, then any move normally landing on square 4 should immediately count as landing on square 14 instead.
 - because of this, players would never actually “stay” on square 4 at the end of a turn.
- the main difficulty is making sure every row of the transition matrix still adds up to 1, since probabilities cannot just disappear and must be reassigned correctly.
- another difficulty is avoiding chains or loops.
 - for example, if a ladder ends on another ladder, the code could repeatedly “teleport” the player multiple times automatically.
 - extra checks would therefore be needed to make sure the board still behaves properly.

Component B: Snakes and Ladders (2)

(a) completed

(b) code below:

```

1  % We build the matrix P and you are required to use P to calculate the probability of winning by the
2  % end of turn n. Enter your solution in Ans below. Note that sol records the answer for different values of
3  % n and is then used to plot the graph
4
5  % Enter SL below
6  SL = [24 2; 23 3; 18 4; 17 5; 12 6; 11 7]
7
8  % Setting up P
9
10 % Construct P, ignoring the snakes and ladders
11 P = zeros(25);
12 for i=1:25
13     for j=i+1:min(i+6,25)
14         P(i,j) = 1/6;
15     end
16 end
17 for i = 1:6
18     P(25-6+i, 25) = min((i+1)/6, 1);
19 end
20
21 % Adjust for the snakes and ladders
22 no_SL=height(SL);
23 for i= 1:no_SL
24     for j=1:25
25         if j == SL(i,2)
26             P(SL(i,1),j) = 1;
27         else
28             P(SL(i,1),j) = 0;
29         end

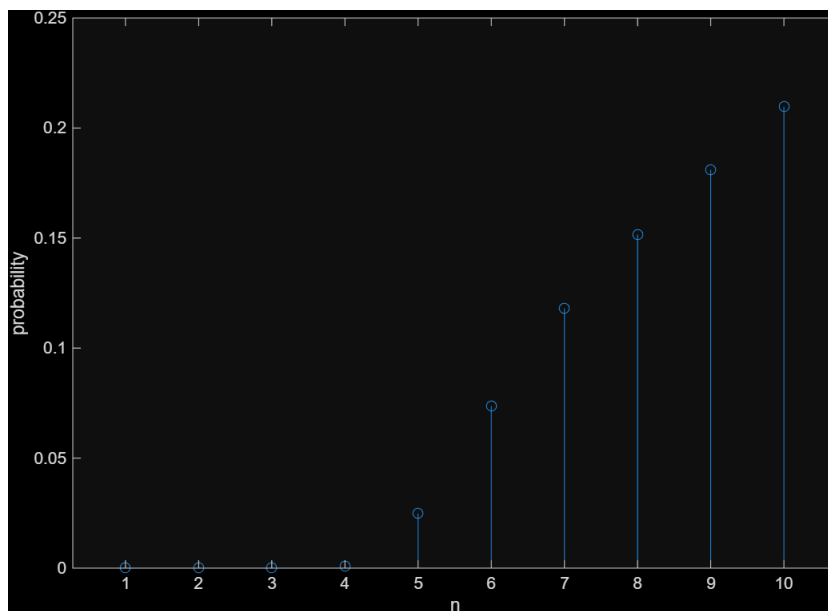
```

```

33 % Record the probability for different values of n
34 sol = [];
35 for n=1:10
36
37     % Your value for the probability of winning by the end of turn n goes below
38     P_n_turns = P^n
39     Ans = P_n_turns(1, 25);
40
41     sol = [sol, Ans];
42 end
43
44 % Plot it out
45 stem(sol);
46 xlabel('n')
47 ylabel('probability')

```

- output



SL =

24	2
23	3
18	4
17	5
12	6
11	7